File System

Virtual File System in C

SPL-1 Midterm Project Presentation



Saadab Rahman Ridmi

Supervisor: Dr. BM Mainul Hossain

Thursday, 12 November 2026

Project Overview

A binary file acts as a small virtual disk

01 Motivation

Understand how storage allocation and persistent data work.

02 Current objective

Manage fixed-size blocks inside virtual_disk.bin.

03 Final outcome

A CLI with named files, directories and path handling.



Methodology

Modular C11 development with a clear project structure

Development approach

Separate responsibilities

CLI, storage logic and disk I/O

Build one feature at a time

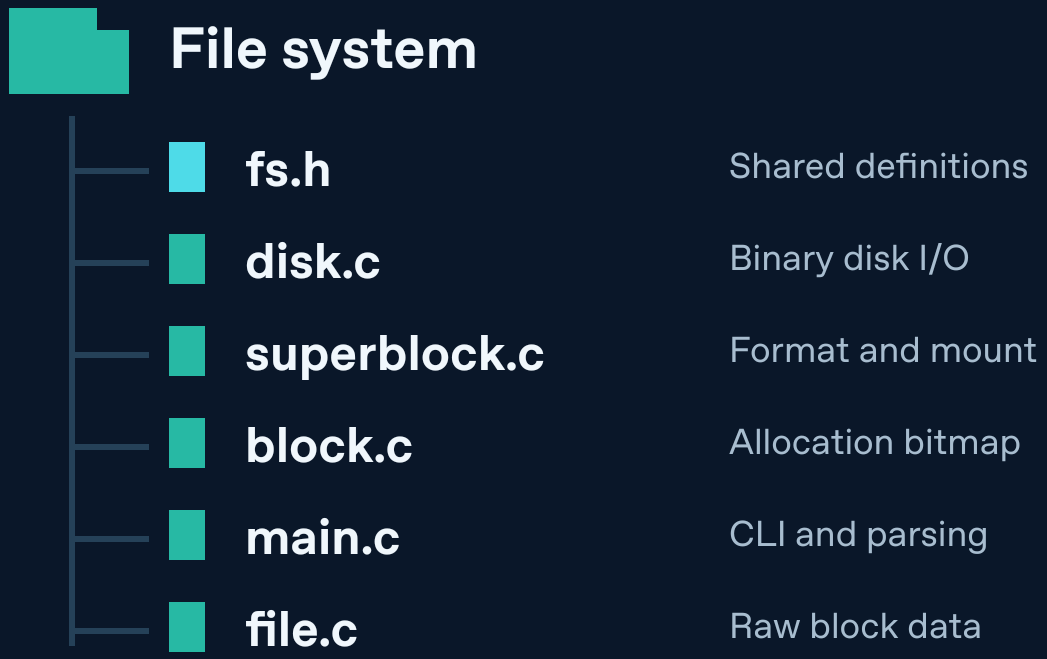
Fixed blocks and first-fit allocation

Inspect and refine

Exercise commands and check metadata

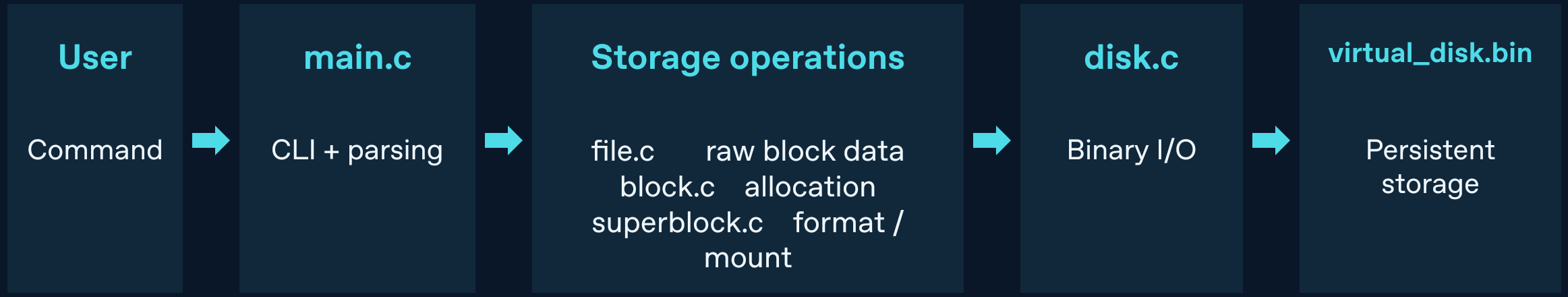
GCC, VS Code, Git and GitHub

Project source tree



System Architecture

Logical flow of a storage command



fs.h Shared constants, structures and declarations

Virtual Disk Layout

1 MiB = 1,048,576 bytes = 2,048 blocks × 512 bytes



Block 0	Blocks 1–4	Blocks 5–36	Blocks 37–2047
1 block	4 blocks	32 blocks	2,011 blocks

Bitmap: one byte per block 0 = free 1 = allocated / reserved

First-fit example

37	38	39	40	41
1: used	0: first free	0: free	0: free	0: free

Illustration: scan from block 37 and choose block 38. Disk segments are schematic.

Implementation Progress and Achievements

Current stage: storage-management prototype

40–45%

Estimated final functional scope

Planning estimate, not a measured completion score



Disk and metadata

1 MiB disk, format / mount commands

Block management

Automatic / specific allocation and release

Raw data operations

Read, write, clear, copy and compare commands

CLI and inspection

Bitmap, layout, statistics and consistency checks

Next: inodes, named files, directories and paths

CLI Commands and Output

Actual terminal evidence from the storage prototype

Allocation and raw data writing

```
fs:blocks> alloc
Allocated data block 39
fs:blocks> allocat 40
Allocated data block 40
fs:blocks> used 40
Block 40: allocated
fs:blocks> used 41
Block 41: free
fs:blocks> writeblock 39 Saadab Rhamna Ridmi
Wrote data to block 39
```

Automatic allocation chose block 39.
Specific allocation reserved block 40.

Allocation bitmap

```
statsBitmap: R = reserved, # = used, . = available
   0-  31  RRRRRRRR RRRRRRRR RRRRRRRR RRRRRRRR
  32-  63  RRRRR... ........ ........ ........
  64-  95  ........ ........ ........ ........
  96- 127  ........ ........ ........ ........
Showing first 128 blocks of 2048.
Data blocks used: 0
Data blocks free: 2011
```

Metadata validation

```
fs:blocks> checksum 39
Block 39 checksum: 866BDE9B
fs:blocks> countbyte 39 8
Byte value 8 occurs 0 times in block 39
fs:blocks> check
Metadata integrity check: PASS
```

Separate captures show different disk states.

Live System Demonstration

2 minutes: allocate, store, inspect and restart

1 Prepare

format
info

2 Allocate + write

alloc
writeblock 37 Hello World

3 Inspect

readblock 37
bitmap
check



exit + restart

readblock 37

Persistence check: same disk file, without formatting

Warning: format erases existing data. Use a disposable demo disk.

Remaining Work and Future Milestones

Four development phases toward the final system

1

Inode management

Initialize the inode table
Allocate / release inodes
Link files to data blocks

2

File management

Create named files
Read, write and append
Delete, stats and block lists

3

Directories and paths

Create / remove directories
Absolute / relative paths
cd, ls, pwd and tree

4

Reliability and integration

Integrate modules
Handle errors and inputs
Test workflows and document

Conclusion and Q&A

A working storage foundation

Persistent virtual disk structure

Block allocation and bitmap management

Next step: inodes and full file operations



Thank you. Questions?